

TRABAJO PRACTICO 1

Materia: Programación II

Lenguaje: JAVA

Profesor: Prof Miguel Silva

Alumnos: Franco Roman Ulrich

Juan Ignacio Ramirez Albamonte

Pereyra, Miguel Adrian

Fecha de presentación: 13 de julio de 2026

SISTEMA TIJETRAVEL

Sistema para gestión de cadena de agencias de viajes

1. Introducción

El sistema TijeTravel fue desarrollado como trabajo practico para Programación II. La consigna solicitaba resolver la gestión de una cadena de agencias de viaje, teniendo en cuenta sucursales, hoteles, vuelos, turistas, reservas y usuarios con distintos permisos.

La aplicación se usa por consola y guarda los datos en archivos de texto. Elegimos mantener esa forma porque era parte del enunciado: no usar interfaz gráfica y no usar base de datos. A partir de eso armamos una estructura con clases separadas para que el programa no quede concentrado en un solo archivo.

La intención del trabajo fue mostrar los conceptos principales de POO en un caso concreto. Por eso no pensamos el código solamente como menús, sino como objetos que representan partes del problema: un hotel, un vuelo, una reserva, un turista, un usuario, etc.

2. Diseño del sistema

El proyecto quedo dividido en cuatro paquetes principales:

- modelos
- controladores
- persistencia
- vistas

En modelos están las clases que representan la información del sistema. Ahí aparecen Agencia, Sucursal, Hotel, Vuelo, Turista, Reserva y los usuarios.

La clase Agencia funciona como punto central porque mantiene las listas principales y permite buscar, agregar o eliminar elementos.

La clase Reserva es una de las que mejor muestra la relación entre objetos. Una reserva no está aislada: tiene un turista, una sucursal, un vuelo y un hotel. También guarda la clase del vuelo, el tipo de hospedaje y las fechas de llegada y partida. Esta estructura se determinó así para respetar lo que pedía el enunciado de la consigna, donde el turista contrata desde una sucursal, viaja en un vuelo y se hospeda en un hotel.

Los controladores quedaron como una capa intermedia entre los menús y el modelo. Por ejemplo, `ControladorReservas` revisa si una reserva se puede crear o modificar antes de agregarla a Agencia. Allí se validan fechas, disponibilidad del vuelo, plazas del hotel y que el destino del vuelo coincida con la ciudad del hotel.

La persistencia está separada en clases propias para cada archivo. El sistema usa archivos como `hoteles.txt`, `vuelos.txt`, `turistas.txt`, `usuarios.txt` y `reservas.txt`. Cada clase sabe como leer su archivo y como volver a guardar los datos.

Las vistas son los menús de consola. `VistaPrincipal` muestra las opciones iniciales y después del login deriva a `VistaCliente`, `VistaVendedor` o `VistaAdministrador`, según el rol del usuario que ingreso.

3. Decisiones de implementación

Una decisión importante fue separar los usuarios por rol. No era suficiente tener una sola clase con un campo de texto, porque cada tipo de usuario podía hacer cosas distintas. Por eso creamos Usuario como clase abstracta, y desde allí salen Cliente, Vendedor y Administrador.

Para los roles y permisos usamos enumeradores.

- RolUsuario tiene CLIENTE, VENDEDOR y ADMINISTRADOR.
- Permiso tiene acciones como consultar, administrar reservas, administrar clientes o administrar vuelos.

Esto nos pareció más ordenado que comparar textos escritos a mano en varios lugares.

También usamos UsuarioFactory para crear usuarios. Esto se definió para centralizar la decisión de que clase concreta corresponde construir según el rol. Si el usuario cargado es CLIENTE, se crea un Cliente; si es VENDEDOR, un Vendedor; y si es ADMINISTRADOR, un Administrador.

Para las fechas usamos LocalDate y LocalDateTime. Esto nos ayudó porque las reservas necesitan comparar llegada y partida, y los vuelos tienen fecha y hora. Si todo se manejaba como texto, las validaciones quedaban más incómodas.

En Agencia las listas internas son ArrayList, pero no se devuelven directamente para que cualquiera las cambie. Los métodos de consulta devuelven listas no modificables. Para agregar o borrar datos hay que usar métodos de Agencia, donde se pueden hacer controles como no repetir códigos o no borrar elementos que ya tienen reservas.

4. Uso de POO en el trabajo

- Abstracción

La abstracción aparece en Usuario. En muchas partes del sistema no importa si el usuario es cliente, vendedor o administrador; alcanza con saber que es un Usuario y que puede responder su rol y sus permisos.

También aparece en Archivo<T>. Esa interfaz representa la idea general de un archivo que puede cargar y guardar una lista. Después cada clase concreta resuelve el detalle:

ArchivoHoteles carga hoteles, ArchivoVuelos carga vuelos, ArchivoUsuarios carga usuarios, y así con los demás.

- Encapsulamiento

Los atributos de las clases del modelo son privados. Para consultar se usan getters, y para modificar datos se usan métodos como actualizarDatos. Esto se ve en Hotel, Vuelo, Turista, Sucursal y Reserva.

En lugar de permitir cambios directos, esos métodos revisan algunas condiciones. Por ejemplo, un vuelo no debería tener plazas negativas, y una reserva no debería actualizarse con objetos nulos.

También aplicamos encapsulamiento en Agencia. Las listas principales no se modifican desde afuera de cualquier manera. Eso evita que otra parte del programa agregue datos repetidos o deje el sistema en un estado indefinido.

- Herencia

La herencia se usa en los usuarios:

Cliente hereda de Usuario.

Vendedor hereda de Usuario.

Administrador hereda de Usuario.

Comparten nombre de usuario y contraseña, pero cada uno define su rol y sus permisos.

También hay herencia en las vistas. VistaCliente, VistaVendedor y VistaAdministrador heredan de VistaUsuario, porque todas necesitan trabajar con la agencia, los controladores y el scanner de consola.

En persistencia, las clases de archivos reutilizan comportamiento de ArchivoTexto. Eso evita repetir código para abrir archivos, separar campos o guardar de forma similar.

- Polimorfismo

El polimorfismo se ve principalmente con el método tienePermiso. El sistema puede tener una variable de tipo Usuario, pero el resultado cambia según el objeto real.

Un Cliente solo puede consultar. Un Vendedor puede consultar y administrar clientes y reservas. Un Administrador puede hacer todas las acciones. La llamada al método es la misma, pero la respuesta depende de la clase concreta.

También se ve algo parecido en las vistas: todas tienen un método mostrar, pero cada vista muestra un menú distinto según el rol.

- Relaciones entre objetos

- Además de herencia, el trabajo usa asociaciones entre clases. La Reserva se relaciona con Turista, Sucursal, Vuelo y Hotel. Agencia agrupa las colecciones principales del sistema. Estas relaciones ayudan a que el código se parezca más al problema real que estábamos modelando.

5. Aplicación de principios S.O.L.I.D.

No todos los principios quedaron aplicados de forma perfecta, pero intentamos tenerlos presentes al ordenar el proyecto.

- Responsabilidad única: Separamos el sistema para que cada parte tenga un objetivo claro. Las vistas muestran menús, los controladores validan operaciones, los modelos representan los datos y persistencia se ocupa de leer y guardar archivos. Un ejemplo concreto es `ControladorReservas`. En vez de hacer todas las validaciones dentro del menú, las fechas, cupos de hotel, cupos de vuelo y compatibilidad entre vuelo y hotel son administradas por el controlador.
- Abierto / cerrado: La jerarquía de Usuario ayuda en este punto. Si hubiera que agregar otro tipo de usuario, se podría crear otra subclase con sus permisos. Igual hay una limitación: también habría que tocar `RolUsuario` y `UsuarioFactory`. Para este trabajo lo aceptamos porque la consigna ya definía tres tipos de usuarios y no era necesario preparar un sistema mucho más grande.

- Sustitución de Liskov: Cliente, Vendedor y Administrador pueden usarse donde se espera un Usuario. Los tres respetan la misma idea base: tienen credenciales, rol y permisos. Por eso ControladorAutorizacion no necesita saber todos los detalles del usuario. Le pregunta si tiene un permiso y cada clase responde segun corresponda.
- Segregación de interfaces: La interfaz Archivo<T> es chica. Solo pide cargar y guardar. No obliga a las clases a implementar operaciones que no usan. Esto nos pareció adecuado porque los archivos del sistema necesitaban justamente eso: leer datos al iniciar y guardar cambios al finalizar o después de una operación.
- Inversión de dependencias: Este principio quedo aplicado solo en algunas partes. Usuario y Archivo<T> funcionan como abstracciones. Sin embargo, varios controladores dependen directamente de Agencia y las vistas crean algunos controladores. Para una versión mas avanzada se podría mejorar usando mas interfaces o pasando dependencias desde afuera. Para el alcance de una aplicación de consola, priorizamos que el código fuera entendible y no demasiado excesivo.

6. Validaciones que agregamos

Además de cargar y mostrar datos, el sistema controla varias situaciones para que las operaciones tengan sentido:

- No se puede crear una reserva con fecha de llegada posterior o igual a la de partida.
- No se permite reservar un vuelo si ya no quedan plazas en la clase elegida.
- No se permite reservar un hotel si no tiene plazas disponibles en esas fechas.
- Un turista no puede reservar dos veces el mismo vuelo.
- El hotel elegido tiene que estar en la misma ciudad que el destino del vuelo.
- No se pueden borrar hoteles, vuelos o sucursales que ya estén usados en reservas.
- No se puede eliminar el ultimo administrador.

Estas reglas están repartidas principalmente entre Agencia y ControladorReservas. Nos pareció importante agregarlas porque si no el sistema quedaba como una simple carga de datos, sin respetar las condiciones del problema.

7. Persistencia

La información se guarda en archivos de texto dentro de TijeTravel/datos. Cada línea representa un registro y los campos se separan con punto y coma.

ControladorDatos se encarga de cargar todo al inicio y guardar todo cuando corresponde. El orden de carga no es casual: primero se cargan los datos base, como sucursales, hoteles, vuelos, turistas y usuarios. Después se cargan las reservas, porque para armar una reserva hay que encontrar antes el turista, el hotel, la sucursal y el vuelo.

La clase ArchivoTexto concentra operaciones comunes, como abrir lectores, separar campos y guardar usando un archivo temporal. Con esto evitamos repetir la misma lógica en cada archivo.

8. Aclaraciones y mejoras posibles

El sistema esta pensado como trabajo académico. Cumple con la consigna y muestra los temas de la materia, pero hay cosas que en una aplicación real se tendrían que mejorar.

La más clara es la seguridad de las contraseñas. En este proyecto se guardan en texto plano en usuarios.txt. Para un sistema real habría que usar encriptación y otras medidas de seguridad.

Otra mejora posible seria ordenar aún mas las vistas. Al ser menús de consola, algunas clases tienen bastante código de lectura y muestra de opciones. Funciona para este trabajo, pero si el sistema creciera convendría separar mejor esa parte.

También se podría reemplazar la persistencia en archivos por una base de datos. En este caso no se hizo porque la consigna aclaraba persistencia sin base de datos.

9. Conclusión

El proyecto permite administrar desde consola los datos principales de una agencia de viajes: sucursales, hoteles, vuelos, turistas, reservas y usuarios. Además diferencia permisos entre clientes, vendedores y administradores.

Como grupo, lo mas importante fue aplicar POO en algo concreto. Usamos abstracción, encapsulamiento, herencia y polimorfismo dentro del código, no solamente como conceptos

teóricos. También intentamos aplicar S.O.L.I.D. de una forma razonable para el nivel y el alcance del trabajo.

10. Repositorio Git Hub: <https://github.com/migmik/TPPrograII>

11. UML en formato SVG: [https://insptutneduar-](https://insptutneduar-my.sharepoint.com/personal/miguel_pereyra_alu_inspt_utn_edu_ar/_layouts/15/guestaccess.aspx?share=IQAk2zNj8v5QLojEoFq2ej3AUKdPiw3cNjasy0FF2MMJlk)

[my.sharepoint.com/personal/miguel_pereyra_alu_inspt_utn_edu_ar/_layouts/15/guestaccess.aspx?share=IQAk2zNj8v5QLojEoFq2ej3AUKdPiw3cNjasy0FF2MMJlk](https://insptutneduar-my.sharepoint.com/personal/miguel_pereyra_alu_inspt_utn_edu_ar/_layouts/15/guestaccess.aspx?share=IQAk2zNj8v5QLojEoFq2ej3AUKdPiw3cNjasy0FF2MMJlk)

12. UML en formato PDF: [https://insptutneduar-](https://insptutneduar-my.sharepoint.com/personal/miguel_pereyra_alu_inspt_utn_edu_ar/_layouts/15/guestaccess.aspx?share=IQAW6VmQp4iaRYRpcGjslOJFAbJAS6S6k8bH20dhJMcGHYA&e=kenlH9)

[my.sharepoint.com/personal/miguel_pereyra_alu_inspt_utn_edu_ar/_layouts/15/guestaccess.aspx?share=IQAW6VmQp4iaRYRpcGjslOJFAbJAS6S6k8bH20dhJMcGHYA&e=kenlH9](https://insptutneduar-my.sharepoint.com/personal/miguel_pereyra_alu_inspt_utn_edu_ar/_layouts/15/guestaccess.aspx?share=IQAW6VmQp4iaRYRpcGjslOJFAbJAS6S6k8bH20dhJMcGHYA&e=kenlH9)